NFH-Kuza-Google Platform

Architecture Specification — Rural Service Delivery Marketplace

Domain Code: **local-services**

Version: 1.0

Date: April 2026

Status: Draft

1. Project Overview

NFH-Kuza-Google is a rural service delivery marketplace that connects rural households with local service providers through WhatsApp and a lightweight web application. The platform is built on the **Beckn open network protocol**, enabling decentralized discovery and fulfillment of services across four domains: livelihood services, household services, market produce trading, and agricultural advisory.

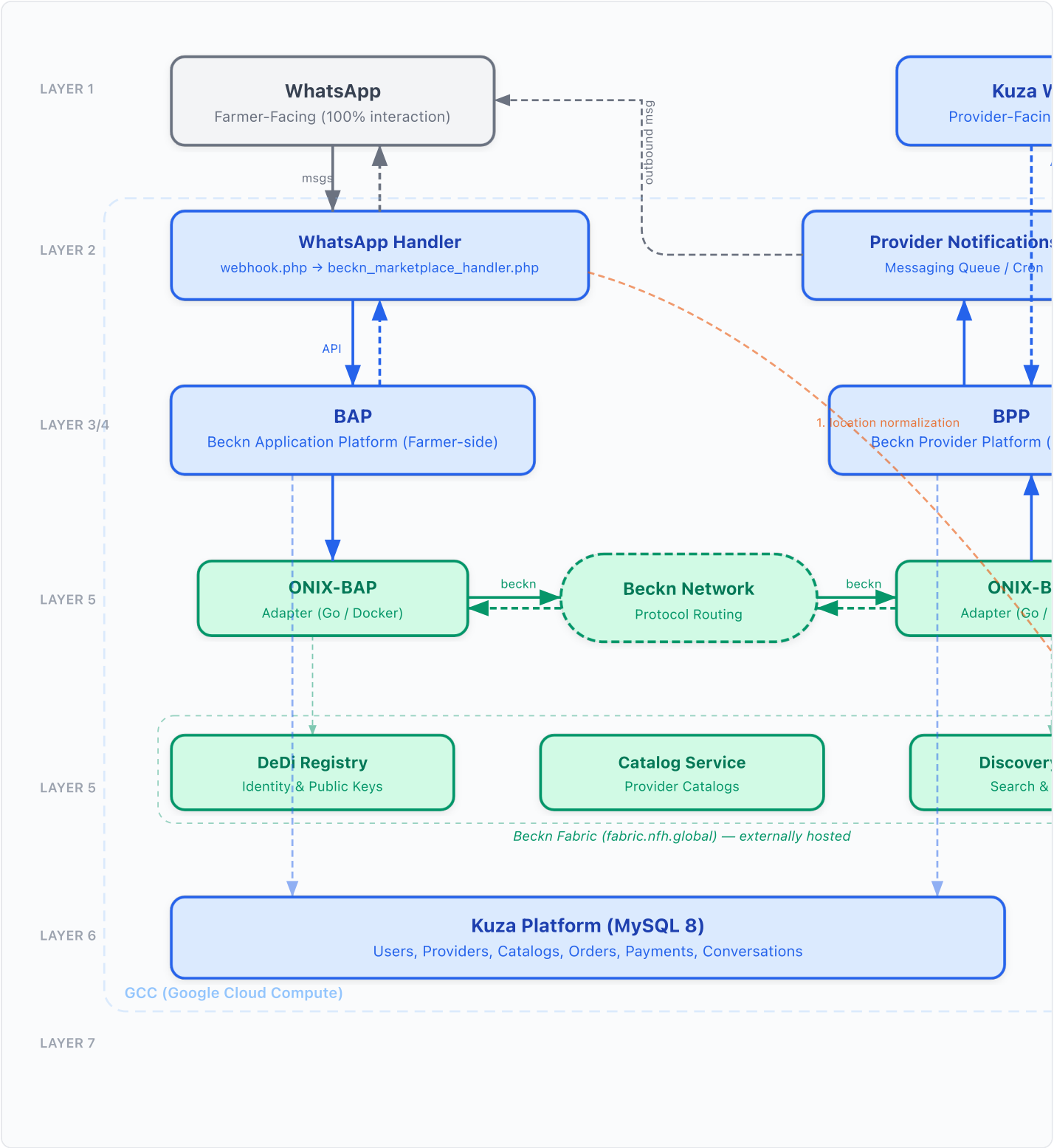
> **Design Principle**
>
> 100% of farmer interaction occurs through WhatsApp — no app installation required. Providers use a lightweight Ionic Angular web app opened via WhatsApp button links.

Service Domains

- **Livelihood Services** — agricultural inputs, equipment rental, veterinary services, soil testing, etc.
- **Household Services** — plumbing, electrical, cleaning, construction, etc.
- **Market Produce** — connecting farmers with aggregators and buyers (reversed economics: buyer pays farmer)
- **Advisory** — agricultural content and guidance delivered as media through WhatsApp (no order lifecycle)

2. Architecture Diagram

Kuza-built components · Beckn infrastructure · Google services · WhatsApp / external



3. Architecture Layers

3.1 Layer 1: Frontend Interfaces

WhatsApp (Farmer-Facing)

WhatsApp is the sole farmer-facing interface. 100% of farmer interactions — service discovery, provider selection, order confirmation, payment declaration, and feedback — occur through WhatsApp messages.

- **Interactive buttons** for small binary/ternary choices (Accept/Reject, Yes/No, payment method selection)
- **Numbered text lists** for longer option sets (provider lists, service categories, sub-categories)
- **Free text input** for location, quantity, and other open-ended fields
- Connected via **WhatsApp Cloud API**

Kuza Web App (Provider-Facing)

A lightweight Ionic Angular web application opened from WhatsApp button links — no native app installation required. Providers use it to:

- Manage their service catalog (add/edit services, set pricing)
- View and accept/reject incoming orders
- Update order status (in-progress, completed)
- Report payment received
- Manage their profile and service areas

3.2 Layer 2: WhatsApp Handler

> **Design Constraint**
>
> The existing `webhook.php` remains UNTOUCHED. Only routing logic is added to branch marketplace conversations to the new `beckn_marketplace_handler.php` based on trigger words or message IDs.

- **File:** `beckn_marketplace_handler.php` (new PHP file on Kuza Platform)
- **State machine:** manages conversational flow for the 4 marketplace domains (Livelihood, Household, Market, Advisory)
- **Translates** WhatsApp interactions into BAP API calls
- **Receives** async callbacks from BAP and formats WhatsApp response messages
- **Own tables:** conversation state, session tracking — separate from existing poultry chatbot tables (`chatbot_nodes`, `chatbot_transitions`)
- **Provider notifications** are triggered via a messaging queue/cron job, NOT through `webhook.php`

3.3 Layer 3: BAP (Beckn Application Platform)

The BAP is the farmer-side Beckn participant. It acts as the bridge between the WhatsApp Handler and the Beckn network.

- Receives translated requests from the WhatsApp Handler
- Constructs Beckn v2 protocol messages: `discover`, `select`, `init`, `confirm`, `status`, `cancel`, `rate`
- Sends protocol messages through the ONIX-BAP adapter
- Receives async callbacks (`on_discover`, `on_select`, `on_init`, `on_confirm`, `on_status`, `on_cancel`, `on_rate`)
- Pushes results back to WhatsApp Handler for farmer delivery
- Hosted by Kuza on GCC

3.4 Layer 4: BPP (Beckn Provider Platform)

The BPP is the provider-side Beckn participant. It manages the provider catalog, order lifecycle, and serves the provider web interface.

- Receives Beckn protocol requests from the network (via ONIX-BPP adapter)
- Connects to Kuza Platform to fetch provider catalogs, service areas, and pricing
- Manages order lifecycle on the provider side (create, accept, update status, complete)
- Sends async Beckn responses back through ONIX-BPP adapter
- Serves the **Kuza Web App** (provider interface) for catalog and order management
- Hosted by Kuza on GCC

3.5 Layer 5: Beckn Network Infrastructure

All Beckn network components are external to Kuza infrastructure, provided by Beckn Fabric.

ONIX Adapters

- Plugin-based Go servers running the `fidedocker/onix-adapter` Docker image
- Two instances: ONIX-BAP (farmer side) and ONIX-BPP (provider side)
- Handle message signing, signature validation, schema validation, and network routing
- Hosted on GCC as Docker containers

DeDi Registry

- Endpoint: `fabric.nfh.global/registry/dedi`
- Stores participant identity, public keys, and callback URIs
- Used for signature validation and dynamic routing between BAP and BPP

Catalog Service

- Endpoint: `fabric.nfh.global/beckn/catalog`
- BPP publishes provider catalogs here
- Queried by Discovery Service during `discover` requests

Discovery Service

- Handles `discover` requests from BAP
- Queries Catalog Service for matching providers
- Returns results to BAP via `on_discover` callback

3.6 Layer 6: Kuza Platform (Data Store)

MySQL 8 on GCC serves as the single source of truth. The Beckn network is stateless — all persistent data lives here.

Existing Tables Used

- `user` — agripreneurs (providers) and farmers
- Partner and organization data
- Content library (for advisory domain)

New Tables Required

Table	Purpose
`beckn_provider_catalog`	Services offered by providers, with pricing per service
`beckn_provider_service_areas`	County + Sub-County mappings for provider coverage areas
`beckn_subcategory_master`	Dynamic taxonomy of service sub-categories
`beckn_orders`	Order records with full lifecycle tracking
`beckn_order_status_history`	Audit trail of status transitions per order
`beckn_payments`	Payment records (farmer-side and provider-side)
`beckn_feedback`	Feedback and ratings from both parties
`beckn_conversation_state`	Marketplace conversation state machine data
`beckn_farmer_profile`	One-time farmer profile fields (location, preferences)

3.7 Layer 7: Google AI Services

Google AI is integrated directly into the request processing pipeline — not a standalone service. It is called from the PHP handler and the response flows back through the same handler to WhatsApp.

- **Location normalization (all flows):** Farmer enters free-text location (village/ward/county) → PHP handler sends text to Google AI → AI maps to structured County + Sub-County → PHP handler sends confirmation message to farmer on WhatsApp → farmer confirms or corrects.
- **Advisory content generation (Advisory flow):** Farmer selects Advisory → picks domain (Agri, Health, Finance) → category → sub-category → these attributes are sent from the PHP handler to Google AI → AI processes the request and generates the advisory response → PHP handler receives the AI-generated content → sends it back to farmer on WhatsApp. This flow goes through Beckn (BAP → BPP → Google AI → BPP → BAP) with Google AI called from the BPP side.
- Exposed as REST APIs on GCC
- Called synchronously from the PHP handler during request processing
- Co-located on GCC

4. Key Data Flows

Flow A: Farmer Discovers and Books a Service (Livelihood / Household)

```
Farmer (WhatsApp): Selects category --> service --> enters quantity
  --> webhook.php --> beckn_marketplace_handler.php
    Farmer types location (free text, e.g. "Naivasha")
    --> PHP Handler calls Google AI for location normalization
    <-- Google AI returns {county: "Nakuru", subCounty: "Naivasha"}
    --> WhatsApp: "Nakuru County, Naivasha Sub-County. Correct?" [Yes] [No]
    Farmer confirms [Yes]
    --> BAP Client --> ONIX-BAP --> [Beckn Network / Discovery Service] --> ONIX-BPP -->
BPP
      --> Kuza Platform (query providers by location + category + subCategory)
    <-- BPP --> ONIX-BPP --> [Beckn Network] --> ONIX-BAP --> BAP Client
  <-- beckn_marketplace_handler.php --> WhatsApp (list of providers with prices)

(continues through select / init / confirm / status / update / cancel / rate / support —
 see Section 4.6 for the complete Beckn endpoint mapping covering the full order
lifecycle.)
```

Flow B: Provider Receives and Accepts Order

```
Order confirmed in Beckn
  --> BPP --> Kuza Platform (create order record)
  --> Messaging Queue / Cron --> WhatsApp Cloud API --> Provider WhatsApp
    Provider taps [Accept] button in WhatsApp
      --> webhook.php --> provider routing --> BPP --> Beckn update
    Provider taps [View Details] link
      --> Opens Kuza Web App in browser --> reads from Kuza Platform
```

Flow C: Provider Completes Service, Farmer Confirms

```
Provider (Kuza Web App):
  --> Marks service complete, enters payment details, notes
  --> BPP --> Beckn status update --> ONIX-BAP --> BAP
  --> beckn_marketplace_handler.php --> WhatsApp to Farmer
    "Was the service completed?" [Yes] [No]
  Farmer taps [Yes]
  --> webhook.php --> beckn_marketplace_handler.php
    --> "How did you pay?" [Cash] [M-Pesa]
  Farmer selects payment method
  --> Order marked FARMER_CONFIRMED --> CLOSED
```

Flow D: Farmer Gets Advisory Content (Google AI Integrated)

```
Farmer (WhatsApp): Selects Advisory --> Domain (Agri/Health/Finance) --> Category -->
Sub-category
  --> webhook.php --> beckn_marketplace_handler.php
    --> BAP Client --> ONIX-BAP --> [Beckn Network] --> ONIX-BPP --> BPP
      --> BPP calls Google AI with {domain, category, subCategory, farmerContext}
      <-- Google AI returns generated advisory content
    <-- BPP --> ONIX-BPP --> [Beckn Network] --> ONIX-BAP --> BAP Client
  <-- beckn_marketplace_handler.php --> WhatsApp (AI-generated advisory message/content)

Google AI is called from BPP during on_discover processing.
Advisory is free, no order lifecycle. Content generated and delivered immediately.
```

Flow E: Market Produce (Farmer Sells)

```
Same flow as A but with reversed economics:
- Farmer has produce to sell (quantity + type)
- discover returns aggregators/buyers with buying prices
```

```
  - Aggregator collects produce from farmer
  - Aggregator pays farmer (not farmer pays provider)
  - Both parties confirm and provide feedback

  (continues through the same full Beckn lifecycle —
   see Section 4.6 for the complete endpoint mapping including cancellation & feedback.)
```

4.6 Complete Beckn Endpoint Mapping

This section maps every user-facing step in each flow to the corresponding Beckn v2 protocol endpoint, covering the **full order lifecycle** — discovery, selection, initialization, confirmation, fulfilment status, in-flight updates, cancellation, rating/feedback, and support.

Beckn endpoints reference:

- `discover` / `on_discover` — find services/providers in the network
- `select` / `on_select` — request a priced quote for a specific item
- `init` / `on_init` — submit billing & fulfilment details, receive draft order
- `confirm` / `on_confirm` — create the binding order
- `status` / `on_status` — query or push the current state of the contract
- `update` / `on_update` — mutate an existing contract (e.g. reschedule, report payment, add notes)
- `cancel` / `on_cancel` — cancel an order (before or after provider acceptance)
- `rate` / `on_rate` — submit ratings / feedback on provider or farmer
- `support` / `on_support` — request customer support details

> **Conventions used in the tables below:**
>
> - **BAP → BPP** means the call originates at the BAP (farmer side) and is routed through ONIX / Beckn Network to the BPP (provider side). The corresponding `on_*` callback travels in the opposite direction.
> - **status** calls can be *pull* (BAP → BPP to query) or *push* (BPP → BAP as a notification). Both are used.
> - Status values referenced come from `LocalServiceFulfillmentAttributes.serviceStatus`: `ORDER_PLACED`, `PROVIDER_ACCEPTED`, `PROVIDER_REJECTED`, `IN_PROGRESS`, `COMPLETED`, `FARMER_CONFIRMED`, `CANCELLED`, `CLOSED`.

4.6.1 Livelihood Services — Complete Lifecycle

Example: Poultry vaccination, feed supply, brooding, incubation, trades & skills.

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
Discovery	Farmer picks category, sub-category, service, quantity, location, preferred date	`discover`	BAP → BPP	Intent: category=Livelihood-Services, subCategory, quantityRequired, farmerLocation, scheduledDate
	System returns providers matching service area + service	`on_discover`	BPP → BAP	Catalog with providers, items (LivelihoodItemAttributes), offers (LocalServiceOfferAttributes) filtered by service area
Selection	Farmer selects one provider from the list	`select`	BAP → BPP	providerId, itemId, offerId, quantityRequired
	BPP returns final quote + cancellation policy	`on_select`	BPP → BAP	Final priceAmount (unit price × quantity), cancellationPolicy, confirmed offer terms
Initialization	Farmer provides contact details (auto from WhatsApp profile) and scheduled date	`init`	BAP → BPP	farmer contact, farmerLocation, scheduledDate, billing reference
	BPP returns draft order with payment terms	`on_init`	BPP → BAP	Draft order, payment terms (acceptedPaymentMethods: CASH/MPESA), total
Confirmation	Farmer taps [Confirm] on WhatsApp summary	`confirm`	BAP → BPP	Confirmation with consent flag
	Order created, provider notified	`on_confirm`	BPP → BAP	orderId, provider details, serviceStatus: ORDER_PLACED
Provider Response	Provider taps [Accept] in WhatsApp notification	`status` (push)	BPP → BAP	serviceStatus: PROVIDER_ACCEPTED, confirmedVisitTime
	Farmer receives "Provider accepted. Visit on <date>. Contact: <phone>"	*(push notification, no new call)*	—	Farmer notified on WhatsApp

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
	Provider taps [Reject] instead	`status` (push)	BPP → BAP	serviceStatus: PROVIDER_REJECTED → CLOSED; farmer re-presented with alternative providers
	Farmer checks order status manually	`status` (pull)	BAP → BPP	Query by orderId; returns `on_status` with current state
Mid-flight Changes	Provider or farmer changes date, quantity, time slot	`update`	BAP ↔ BPP (either side)	Updated scheduledDate / quantityRequired / confirmedVisitTime
	Other side confirms the change	`on_update`	Opposite direction	Revised order
Service Execution	Provider marks service in progress (via Kuza Web App)	`status` (push)	BPP → BAP	serviceStatus: IN_PROGRESS
	Provider completes service: enters actualServiceDate, payment method, amount received, notes, follow-up flag	`update` + `status`	BPP → BAP	serviceStatus: COMPLETED; providerAmountReceived, providerPaymentReceived: true, providerNotes, followUpRequired; on_update then on_status
Farmer Confirmation	Farmer receives "Was the service completed?" [Yes] [No]	*(WhatsApp; no Beckn call yet)*	—	—
	Farmer taps [Yes] → "How did you pay?" [Cash] [M-Pesa]	`update`	BAP → BPP	serviceConfirmedByFarmer: true, farmerPaymentMethod, farmerPaymentStatus: PAID
	System moves to FARMER_CONFIRMED → CLOSED	`on_update` + `status`	BPP → BAP	serviceStatus: FARMER_CONFIRMED → CLOSED
Cancellation	Farmer cancels before PROVIDER_ACCEPTED (always free)	`cancel`	BAP → BPP	cancellationReason, orderId

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
	Farmer cancels after PROVIDER_ACCEPTED (within cancellation window)	`cancel`	BAP → BPP	cancellationReason; BPP applies cancellationPolicy (refundable flag, window hours)
	Provider cancels (e.g., cannot service)	`cancel`	BPP → BAP	cancellationReason; farmer re-presented with alternatives
	Counterparty acknowledges	`on_cancel`	Opposite direction	serviceStatus: CANCELLED → CLOSED; refund details if applicable
Feedback / Rating	Farmer rates provider (1-5) + selects feedback option + optional text	`rate`	BAP → BPP	target: providerId, rating, selectedOption, additionalText (farmerFeedback)
	Provider rates farmer (cooperation, timeliness, payment)	`rate`	BPP → BAP	target: farmerId, rating, selectedOption, additionalText (providerFeedback)
	Acknowledgement	`on_rate`	Opposite direction	Feedback recorded confirmation
Support	Farmer requests provider contact / help	`support`	BAP → BPP	ref: orderId
	BPP returns contact info	`on_support`	BPP → BAP	contactInfo: phone / email / WhatsApp

4.6.2 Household Services — Complete Lifecycle

Example: School fee payment, hospital checkup, insurance enrolment, consumer goods delivery.

Structurally **identical** to Livelihood mapping (4.6.1). The Beckn endpoint sequence and semantics are the same. Category-specific differences are in the payload fields of `on_discover` / `on_select`:

- `HouseholdItemAttributes` replaces `LivelihoodItemAttributes`
- `subCategory` values: Education, Health, Insurance, Consumer Goods
- `serviceMode` enum: SERVICE_DELIVERY / PRODUCT_DELIVERY / FACILITY_VISIT / DIGITAL
- `beneficiaryType` (INDIVIDUAL / HOUSEHOLD / CHILD / ELDERLY) and `documentsRequired` may be populated

- Some services (e.g., school fee payment) may have `scheduledDate` as the payment deadline rather than a visit date

All lifecycle phases — discovery, selection, init, confirm, status, update, cancel, rate, support — apply the same way. The full Livelihood table above applies to Household with the above payload differences.

4.6.3 Market Produce — Complete Lifecycle (Reversed Economics)

Example: Farmer sells eggs, milk, farm produce to an aggregator / buyer. Money flows provider → farmer.

The Beckn endpoint sequence is the **same as Livelihood**. The semantic reversal is in the payload content, not in the protocol flow. Highlights:

Phase	Beckn Call	Semantic Reversal (vs Livelihood)
Discovery	`discover` / `on_discover`	Farmer advertises produce (`quantityAvailable` , `produceCategory`); BPP returns aggregators/buyers with `buying` prices and `collectionMode` (FARM_GATE_PICKUP / COLLECTION_POINT / FARMER_DELIVERS)
Selection	`select` / `on_select`	Quote is the **buying price per unit** aggregator offers for farmer's produce × quantity
Init / Confirm	`init` , `confirm` (and `on_*`)	Identical protocol, order created to pick up produce at farmer's location / collection point
Provider acceptance	`status` (push)	Aggregator accepts the collection request; serviceStatus: PROVIDER_ACCEPTED
Fulfilment	`status` / `update`	Aggregator collects produce, grades it (`acceptedGrades`), pays farmer; populates `providerPaymentReceived` to indicate *aggregator paid farmer* (NOT farmer paid provider). `amountReceived` = amount paid to farmer.
Farmer confirmation	`update`	Farmer confirms produce was collected and reports method received (Cash / M-Pesa); `farmerPaymentStatus: PAID` = farmer confirms they received payment
Cancellation	`cancel` / `on_cancel`	Either side can cancel before collection. After partial collection, use `update` to adjust quantity actually collected rather than full cancel.
Rate	`rate` / `on_rate`	Farmer rates aggregator's fairness (grading, price honoured, payment promptness); aggregator rates produce quality & farmer reliability
Support	`support` / `on_support`	Same contract as Livelihood

4.6.4 Advisory Content — Discover-Only Lifecycle

Example: Farmer requests advisory on poultry care, financial literacy, health & nutrition.

Advisory is **free, has no order, no payment, no scheduling**. Beckn endpoints used are limited to discovery and optional feedback.

Phase	User-Facing Step	Beckn Call	Direction	Payload Highlights
Discovery with AI Generation	Farmer selects Advisory → domain → category → sub-category	`discover`	BAP → BPP	Intent: category=Advisory-Content, subCategory (topic), farmerContext (location, enterprise, language)
	BPP calls Google AI with the request attributes to generate advisory content	*(BPP-internal; not a Beckn call)*	BPP ↔ Google AI	AI-generated content based on domain / category / sub-category / farmer context
	AI-generated advisory delivered to farmer on WhatsApp	`on_discover`	BPP → BAP	Catalog with AdvisoryContentItemAttributes: contentType, contentUrl / inline text, contentDuration, targetAudience
Feedback (optional)	Farmer answers "Was this helpful?" [Yes] [No] or rates content	`rate`	BAP → BPP	target: contentId, rating (binary helpful/not or 1-5), optional additionalText
	Acknowledgement	`on_rate`	BPP → BAP	Feedback recorded; may trigger content improvement feedback loop
Support (rare)	Farmer asks for more info / clarification on content	`support` / `on_support`	BAP ↔ BPP	Contact info for advisory provider / content author (if applicable); typically not used in pilot
Not applicable	`select`, `init`, `confirm`, `status`, `update`, `cancel` — advisory has no order, so these endpoints are not used.			

Summary of endpoint coverage per flow:

Flow	discover	select	init	confirm	status	update	cancel	rate	support
Livelihood	✓	✓	✓	✓	✓	✓	✓	✓	✓

Flow	discover	select	init	confirm	status	update	cancel	rate	support
Household	✓	✓	✓	✓	✓	✓	✓	✓	✓
Market Produce	✓	✓	✓	✓	✓	✓	✓	✓	✓
Advisory	✓	—	—	—	—	—	—	✓	(rare)

5. Component Ownership

Component	Built By	Hosted On
WhatsApp Handler (`beckn_marketplace_handler.php`)	Kuza	GCC
BAP application logic	Kuza	GCC
BPP application logic	Kuza	GCC
Kuza Web App (provider interface)	Kuza	GCC
Provider WhatsApp notifications	Kuza	GCC
ONIX Adapters (BAP + BPP)	Beckn (open source, Go)	GCC (Docker)
DeDi Registry	Beckn	Beckn Fabric (external)
Catalog Service	Beckn	Beckn Fabric (external)
Discovery Service	Beckn	Beckn Fabric (external)
Google AI (location normalization + advisory content generation)	Google	GCC
MySQL database	Kuza	GCC

6. Database Schema

The following schema additions are required on MySQL 8 (GCC). These tables live alongside existing Kuza Platform tables and reference the existing `user` table via foreign keys.

Entity Relationship Summary

```
user (existing)
  |-- beckn_farmer_profile      (1:1, farmer-specific fields)
  |-- beckn_provider_catalog    (1:N, provider's services)
  |-- beckn_provider_service_areas (1:N, coverage areas)

beckn_subcategory_master        (standalone taxonomy)

beckn_orders
  |-- beckn_order_status_history (1:N, audit trail)
  |-- beckn_payments             (1:N, farmer + provider payments)
  |-- beckn_feedback             (1:N, ratings from both parties)

beckn_conversation_state        (keyed by phone + session)
```

Key Table Definitions

`beckn_orders`

```
order_id          BIGINT AUTO_INCREMENT PRIMARY KEY
order_uuid        VARCHAR(36) UNIQUE NOT NULL
beckn_order_id    VARCHAR(255)        -- Beckn network order ID
farmer_user_id    INT                 -- FK to user.id
provider_user_id  INT                 -- FK to user.id
category          VARCHAR(100)        -- livelihood | household | market | advisory
sub_category      VARCHAR(255)
service_name      VARCHAR(255)
quantity          DECIMAL(10,2)
unit              VARCHAR(50)
quoted_price      DECIMAL(10,2)
currency          VARCHAR(10) DEFAULT 'KES'
status            ENUM('ORDER_PLACED','PROVIDER_ACCEPTED','PROVIDER_REJECTED',
                       'IN_PROGRESS','COMPLETED','FARMER_CONFIRMED',
                       'CANCELLED','CLOSED')
farmer_location   VARCHAR(255)        -- normalized county/sub-county
provider_location VARCHAR(255)
payment_method    VARCHAR(50)         -- cash | mpesa
created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
partner_id        VARCHAR(50)         -- multi-tenancy scope
```

`beckn_conversation_state`

```
id                BIGINT AUTO_INCREMENT PRIMARY KEY
phone_number      VARCHAR(20) NOT NULL
session_id        VARCHAR(100)
current_flow      ENUM('livelihood','household','market','advisory')
current_step      VARCHAR(100)
state_data        JSON                -- accumulated selections, context
beckn_txn_id      VARCHAR(255)        -- Beckn transaction correlation
last_activity     TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
partner_id        VARCHAR(50)
INDEX idx_phone_session (phone_number, session_id)
```

7. Deployment Topology

All Kuza-built components are deployed on Google Cloud Compute (GCC). Beckn Fabric services are externally hosted and accessed over HTTPS.

GCC Instance Layout

Component	Runtime	Port	Notes
Kuza Platform (PHP)	Apache + PHP 7.4	443	Existing platform + WhatsApp Handler + BAP + BPP
Kuza Web App	Nginx (static)	443	Ionic Angular build served as static files
ONIX-BAP	Docker (`fidedocker/onix-adapter`)	5001	BAP-side adapter configuration
ONIX-BPP	Docker (`fidedocker/onix-adapter`)	5002	BPP-side adapter configuration
Google AI APIs	Cloud Functions / API	443	Location normalization + Advisory content generation
MySQL 8	MySQL Server	3306	Single database instance

External Dependencies

Service	Endpoint	Protocol
WhatsApp Cloud API	`graph.facebook.com`	HTTPS REST
DeDi Registry	`fabric.nfh.global/registry/dedi`	HTTPS REST
Beckn Catalog Service	`fabric.nfh.global/beckn/catalog`	HTTPS REST
Beckn Discovery Service	`fabric.nfh.global`	HTTPS REST (Beckn v2)

Deployment Note

ONIX adapters run as Docker containers on the same GCC instance. They require outbound HTTPS access to Beckn Fabric endpoints and inbound access from the BAP/BPP application logic on localhost.